

ChatGPT Tasks (MCP):

從三個系統到一個產品

Week 3 Prototype : Scheduler / LLM Engine / MCP 怎麼串起來

[System_Design_Review]

Demo：用自然語言排一個 Task

「明天早上 8 點幫我總結今天的科技新聞」



「這個系統最像你用過的什麼產品？」

Google Calendar

固定提醒

iOS 捷徑

手動設定 workflow

IFTTT / Zapier

拖拉式自動化

ChatGPT Tasks

自然語言 + LLM 動態執行

Prototype 巡禮

同樣的需求、三條不同的路 — 重點不是誰寫得完整,而是 把哪一面推到了極限。

Gary

[Spec_Driven_AI_Workflow]

把 AI-assisted 開發流程推到極限的人

- 用 **Claude Code superpowers** 跑完整 `spec → plan → execute` 三階段 (`docs/superpowers/{specs,plans}/` , 每 step 有 acceptance criteria 跟 `- [ ]` 追蹤)
- Ops 紀律完整: Docker compose + PostgreSQL + Makefile 自動化 + 嚴格 stderr-only logging(stdio 要 reserve 給 MCP JSON-RPC)
- 差異點是「人類寫 spec、AI 寫 code、checkbox 追蹤」工作流,不是某個 stack

Shelly

[Domain_Extension]

把 prototype 從 toy 變 daily 工具的人

- Schema 加 `job_type` 欄位(generic / `github_pr_check`),MCP tool 支援指定
- 真實 handler: `github_pr_check` 打 GitHub Search API,實際抓 assigned PR + 格式化清單,不只是 "Hello World"
- 把教材 worker 註解「在 `production` 會 call LLM」具體化 — 但用 deterministic handler,不用 LLM

Royce

[Service_Decomposition]

把「教材說要拆開」具象化的人

- **Go** scheduler(HTTP API + Worker + Watcher)+ **Python** MCP server,跨語言透過 HTTP API 互動 + Swagger 文件化
- 拆分讓 scheduler 變成「可被任何 client 呼叫的 service」— MCP 只是其中一種 client(也能接 Web / Mobile / CLI / 其他 LLM agent)
- Docker compose 一鍵起 Postgres + Scheduler + MCP Inspector;5+ 份 design doc(api / db_schema / system_design / watcher / worker)

[Student_Showcase]

設計決策地圖

今天會走的三個決策點

決策一

Job 排程架構的演進

教材直接給了最終版
Live 講怎麼一步步到的

決策二

LLM 怎麼知道該用哪個 tool

Tool selection 的本質
是 prompt engineering

決策三

MCP 為什麼需要存在

為什麼不直接讓
LLM call API

後半段

Code Reality (填空解說) → Production Gap(4 個情境題 + 2 個 teaser)→ 預告 Deep Dive

決策一：Job 排程 — 最簡單的方案

「如果你只有一天時間，怎麼做？」



優點

簡單、一個 process 搞定、容易理解

問題

Cron job 本身掛了怎麼辦？

Job 執行太久，擋住下一輪掃描？

→ querying 和 execution 耦合在一起

「怎麼解耦？」 → 分離 *Watcher* 和 *Worker*

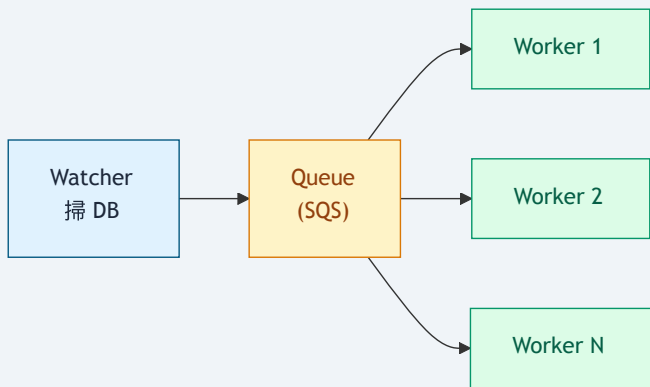
決策一：分離 → 加 Queue → 教材方案

一步步演進到最終架構

第二步：分離 Watcher 和 Worker

Watcher 只負責掃 DB 找到期 job、Worker 只負責執行

但 Watcher 掃出來的 job 怎麼交給 Worker ? 直接 RPC call ?



第三步：加 Queue（教材的方案）

解耦：Watcher 和 Worker 互不知道對方 / Buffer 突發流量 / Failure isolation：一個 Worker 掛了不影響其他

[Evolution_Step2_3]

決策二:LLM 怎麼知道該用哪個 Tool

Tool selection 本質上是 prompt engineering

先釐清:什麼是 Tool?

Tool = 暴露給 LLM 的 function(有 name、description、input schema)。LLM 不直接執行,而是讀完所有 tools -> 決定 call 哪個 + 產生 function call -> 應用層收到後執行 -> 結果回傳 LLM

LLM 看到的東西

```
{
  "tools": [
    { "name": "task.create", "description": "Schedule a new task for future execution" },
    { "name": "task.list", "description": "List all scheduled tasks for the user" }
  ],
  "user_message": "明天早上 8 點幫我總結科技新聞"
}
```

LLM 要做的是「配對」

使用者說的動詞 → 哪個 tool 的 description 最匹配
所以 **tool 命名和描述的品質**直接決定準確率

決策二：Tool 命名的學問

為什麼 *action-verb* 命名這麼重要？

不好的命名

```
createTask  
getTaskStatus  
deleteScheduledItem
```

駝峰式 → LLM 要先「拆解」命名
命名風格不一致 → 增加歧義

好的命名

```
task.create  
task.status  
task.cancel
```

命名空間 + 動詞 → 結構清晰
LLM 在 pre-training 中大量看過 imperative 語句

為什麼 LLM 對 *action-verb* 更敏感？

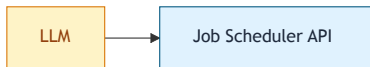
Pre-training 資料中大量的 CLI 指令、API 文件、教學文章都用 imperative 語句

`task.create` 比 `createTask` 更容易被 LLM 正確匹配

決策三：MCP 為什麼需要存在

為什麼不讓 LLM 直接 call Job Scheduler API ?

方案 A：LLM 直接呼叫 API



- API endpoint + auth token 都要放進 prompt
- 不同 LLM provider 各自實作一套
- API 改版 → 每個 LLM 整合都要改

緊耦合

方案 B：透過 MCP



- 標準化介面，任何支援 MCP 的 LLM 都能用
- Backend API 改了只改 MCP server
- 權限控制在 MCP 層做，不用信任 LLM

解耦 + 標準化

本質：MCP 是 LLM 世界的 API Gateway

就像 API Gateway 把 client 和 backend 解耦一樣，MCP 把 LLM 和 backend 解耦

Code Reality 1 : Time Bucket 邏輯

「為什麼用 time_bucket 當 partition key ?」

```
def get_time_bucket(scheduled_at: datetime) → str:
    """Time bucket = DB partition key, 取到小時級"""
    return scheduled_at.strftime("%Y%m%d%H")

def find_due_jobs(current_time: datetime, db: Session) → list[Job]:
    """Watcher 每幾秒呼叫: 找當前 bucket 中到期的 job"""
    bucket = get_time_bucket(current_time)
    return db.query(Job).filter(
        Job.time_bucket == bucket,
        Job.scheduled_at ≤ current_time,
        Job.status == "pending",
    ).all()
```

Partition Key

同小時的 job 放一起 -> 查詢只掃一個 partition

粒度選擇

小時級 vs 分鐘級 -> 粒度太細 partition 太多

對比學生寫法

「你是直接 WHERE scheduled_at ≤ now() 嗎？」

Code Reality 2 : MCP Tool Call Routing

「收到 function call 後怎麼分發到正確的 handler？」

```
# Registry pattern - 新增 tool 只需要加一行
TOOL_REGISTRY = {
    "task.create": handle_create_task,
    "task.list": handle_list_tasks,
    "task.status": handle_get_status,
    "task.cancel": handle_cancel_task,
}

def route_tool_call(tool_name: str, arguments: dict, db: Session) → dict:
    handler = TOOL_REGISTRY.get(tool_name)
    if handler is None:
        return {"error": f"Unknown tool: {tool_name}"}
    return handler(db, **arguments)
```

Registry Pattern

新增 tool = 加一行 dict; 對比 if-else 版本要多 elif + 容易忘 -> 違反 Open-Closed

追問

「加一個新 tool 改幾行？」 Registry: 1 行; if-else: 至少 2 行 elif

Production Gap 1:時區處理

使用者在台灣說「每天早上 8 點」
Server 在 us-east-1，你的 cron 怎麼存？

想 15 秒，打在聊天室

陷阱

- 台灣早上 8 點 = UTC 00:00
- 直接存 UTC cron → `0 0 * * *`
- 但 DST 怎麼辦？美國用戶的「早上 8 點」一年偏移兩次

設計方向

- 存 **本地時間 + timezone ID** (`08:00 Asia/Taipei`)
Timezone ID = IANA 標準地區標識，格式 `Area/City`
- Watcher 掃描時即時轉換成 UTC
- 不要用 offset (+08:00)，要用 timezone ID

為什麼用 timezone ID 不用 offset？

`America/New_York` DST 時 UTC-4，非 DST 時 UTC-5 — 存 offset 夏天就會晚一小時

Production Gap 2: Job 依賴

使用者說「先幫我查匯率，再幫我下單」
Prototype 每個 job 獨立，你怎麼表達先後關係？

想 15 秒，打在聊天室

簡單方案：Job Chain

- Job A 完成後觸發 Job B
- `parent_job_id` 欄位
- Worker 完成 A → 檢查 child job → 排入 queue

進階方案：DAG

- 依賴關係形成有向無環圖
- 類似 Airflow 的 task dependency
- 複雜度高，prototype 不需要

大部分需求是線性的 (A→B→C)，chain 就夠了 — 等有 fan-out / fan-in 再上 DAG，不要過度設計

Production Gap 3: Monitoring

這個系統上線後，你第一個想看的 dashboard 是什麼？

想 15 秒，打在聊天室

Job 健康度

- 成功率 (> 99%)
- 平均執行延遲
- Queue depth (積壓量)

LLM 成本

- 每個 job 的 token 消耗
- 日 / 週 / 月總成本
- Per-user 成本分佈

異常偵測

- 連續失敗 > N 次 → alert
- LLM latency spike
- 用戶突然大量排 job

Prototype 掛了你不曾知道 — production 第一天就要有 monitoring，不是「以後再加」

Production Gap 4:多租戶隔離

一個使用者排了 10 萬個 job，
其他使用者的 job 全部延遲，你怎麼防？

想 15 秒，打在聊天室

限制面

- **Per-tenant quota**：每小時最多 N 個 job
- **Rate limiting**：API 層擋住過量請求
- 超過 quota → 低優先級 queue 或直接拒絕

隔離面

- **Fair scheduling**：round-robin across tenants
- **Per-tenant queue**：每個用戶獨立 queue
- Worker 輪流從不同 queue 取 job

Noisy neighbor — 多租戶經典問題，Quota + Fair scheduling 是最常見的組合

Production Gap 預告:留給下週展開的兩題

這兩題下週 Deep Dive 會用 Black Friday scenario 完整展開

Gap A:Exactly-Once Execution

- 「同一個 job 被執行兩次,使用者看到什麼?」
- 讀取型(總結新聞)/ 寫入型(下單)後果天差地別
- 解法方向:at-least-once + idempotent handler

下週開場 Q3 直接展開

Gap B:LLM 作為不穩定元件

- LLM timeout / 格式錯 / 幻覺 -> 怎麼 retry?
- 跟傳統 API 不同:LLM 200 OK 也可能是垃圾
- 解法方向:Timeout / Validation / Retry / Cost 四策略

下週整堂課的主軸

為什麼留到下週?

這兩題在「日常排程」場景就是問題,但在 Black Friday 50K QPS 下會放大十倍 — 用 scenario 講比抽象 case 深刻得多

下週預告：Deep Dive

Black Friday Scenario

50 萬個 recurring job
同時排在早上 8 點
怎麼辦？

LLM Reliability

LLM 作為不穩定的
系統元件：timeout、
幻覺、cost control

面試實戰

這題怎麼答
45 分鐘的
時間分配建議

課前準備

複習教材的 Deep Dive 章節：at-least-once execution、SQS visibility timeout、MCP 可靠性設計